Chapter 7
Trail planning

The design of a walking or hiking trail is based on fine scale topographic conditions and varied criteria specific to the particular context such as aesthetics, views, construction cost, and environmental sensitivity. As a result trail planning is typically a product of expert knowledge, field surveys, and creative design decisions – often made on site. However, when high resolution data is available geospatial modeling can be used to identify routes optimized for travel time and suitability. To design trails with Tangible Landscape we can hand place waypoints on a physical model and then the optimal network connecting the waypoints is computed in near real-time. This approach – hand placing tangible waypoints and computationally networking the waypoints – combines creative, collaborative decision making with mathematical optimization. In this chapter we explain the theory and methodology for designing trails with Tangible Landscape and then discuss a case study, the design of hiking trail scenarios for Lake Raleigh Woods, North Carolina.

7.1 Trail design methodology

Our approach for designing trail networks with Tangible Landscape combines the creative identification and siting of key waypoints like trailheads and scenic spots with computationally optimized routing between these points. With Tangible Landscape we can place tangible markers by hand to automatically digitize waypoints. This allows us to work intuitively, feel the slopes and curvature with our hands, and easily collaborate.

To find the optimal routes we compute the least cost path between pairs of waypoints over the terrain and a cost surface as a function of walking energetics. First we create a cost surface that represents friction – the additional time required to cross a cell. Next we designate waypoints, points through which we want the trail to pass. For each unique pair of waypoints we use the *r.walk* module to compute the anisotropic cumulative cost of walking

between these points across the terrain and the friction surface based on Naismith's rule for hiking times. Then we compute the least cost path between those points across this cumulative cost surface. All of the least cost paths are combined into a network of potential routes. Finally we solve the traveling salesman problem to identify the best route across this network.

7.1.1 Least cost path analysis

With least cost path (LCP) analysis we can determine the most cost-effective route between a source and destinations on a cost surface. The cost surface is a raster representing cost for traversing a cell and can be derived as a function of distance, slope, land cover or other relevant criteria. To find the least cost path between locations we need to compute a cumulative cost surface where each cell contains the lowest cumulative cost of traversing the space between each cell and the specified location. We also need to generate a movement direction raster tracing the movements that created the cumulative cost surface.

First we compute the walking cost raster using Naismith's rule for walking times (Naismith, 1892) with further adjustments (Aitken, 1977; Langmuir, 1984) of the cost for specific slope intervals. Considering only the elevation surface the cost to walk across a grid cell, expressed as time T in seconds is computed as follows:

$$T = a \cdot \Delta S + b \cdot \Delta H_{\text{uphill}} + c \cdot \Delta H_{\text{moderate downhill}} + d \cdot \Delta H_{\text{steep downhill}} \quad (7.1)$$

where:

- ΔS is the horizontal distance in m
- ΔH is the height difference in m
- a is the time it takes to walk for 1 meter on flat surface in s
- b is the additional walking time per meter of elevation gain on uphill slopes in s
- c is the additional walking time per meter of elevation loss on moderate downhill slopes in s
- d is the additional walking time per meter of elevation loss on steep downhill slopes in s

Up to a specific slope value threshold, walking downhill is faster; after that it becomes more difficult and adds to the time needed to cross the cell. The slope value threshold (slope factor) derived from experiments is -0.2125 corresponding to 12° downslope ($\tan 12° = 0.2125$).

Taking into account land cover conditions the total cost T_{total} in seconds is estimated as a linear combination of movement and friction costs using the dimensionless weight λ:

$$T_{\text{total}} = T + \lambda \cdot F \cdot \Delta S \tag{7.2}$$

where F is the friction in s/m. It represents the additional time in seconds that it takes to walk one meter within a given cell due to its land cover conditions. The friction map may simply be a landcover map that has been recoded as time costs. If landcover data is not available at an appropriate resolution or if there are other important factors that need to be considered, the friction map can be created through map overlay analysis. Parameters such as soils, hydrology, roads, and existing trails can then be incorporated into the cost surface using the friction map.

We used the total cost surface given by Eq. 7.2 to compute a cumulative cost surface where each cell contains the lowest cumulative cost of traversing the space between each cell in the region and the target location. The movement direction raster is also generated tracing the movements that created the cumulative cost surface. We can then compute LCP between any point in the study area and the target location based on the cumulative cost raster and movement directions raster.

7.1.2 Network analysis

Using LCP analysis it is possible to create many combinations of routes between the given waypoints. This can result in a large number of connections and potential trails. We can use network analysis to find a trail loop that goes through all given waypoints in an optimal order, avoiding connections with high costs (Fig. 7.1. The solution for this well-known optimization problem – the Traveling Salesman Problem (TSP)[1] – is implemented in the module *v.net.salesman*, which uses a heuristic algorithm yielding a good solution in a reasonable amount of time.

7.1.3 Trail slope extraction

When planning trails we are typically interested in the average and maximum slope of a trail. However, the extracted values are the slope values in the direction of steepest slope rather than in the direction of the trail. To compute that, we need to know the direction of the trail, which can be derived from its vector representation. The slope along the trail β_t can then be computed as the steepest slope β (see Eq. 4.5) multiplied by the cosine of the angle between the direction of the trail α_t and the steepest slope direction α (i.e. aspect, see Eq. 4.6):

$$\tan \beta_t = \tan \beta \cos(\alpha - \alpha_t) \tag{7.3}$$

The difference is visible in Fig. 7.2 where part of a trail along a contour line has very low slope values along in the trail, but has higher slope values

[1] See: `https://en.wikipedia.org/wiki/Travelling_salesman_problem`

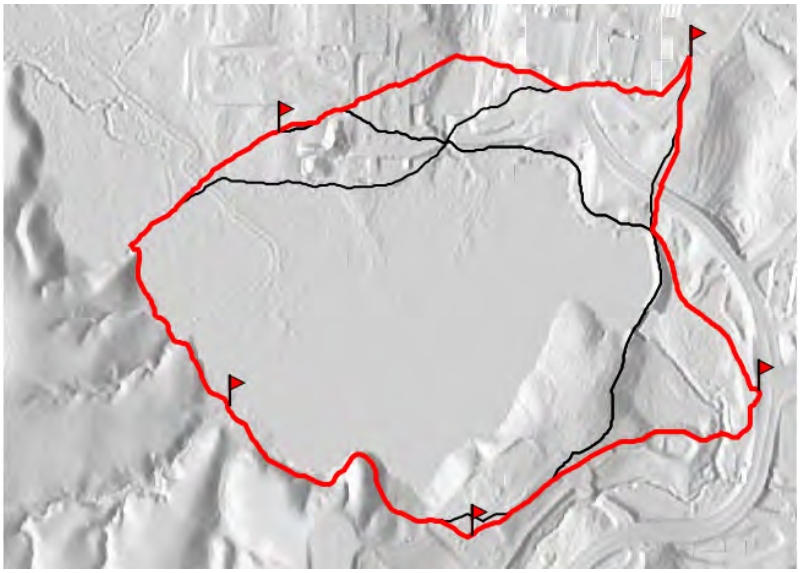
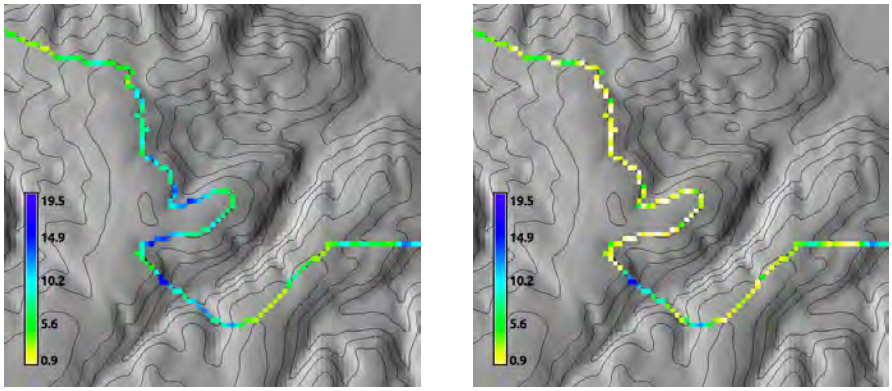


Fig. 7.1: The TSP applied to a network of potential trails. Black lines represent potential routes. The red line is the most cost-effective combination of routes that connects all of the waypoints.

in the direction of the steepest slope. Both types of slope computation can be useful depending upon our objectives. When building and maintaining a trail we may be interested in the steepness of the terrain in that area, while a hiker may be more interested in the directional slope of a trail.

7.2 Case study: designing a recreational trail

We used Tangible Landscape to design alternative trail scenarios for Lake Raleigh Woods on North Carolina State University's Centennial Campus. This old growth woodland on the south shore of Lake Raleigh is rich in biodiversity with approximately 200 species of vascular plants and regionally important stands of mesic mixed hardwood and dry-mesic oak-hickory forest (Blank et al., 2010). Informal hiking and mountain biking trails have caused significant disturbance in the woods and erosion along the lakefront. A formal trail system could concentrate people on low-impact routes and thus conserve the landscape while enhancing opportunities for recreation.



(a) (b)

Fig. 7.2: The difference between the trail slope extracted (a) in the direction of the steepest slope and (b) in the direction of the trail (in degrees).

7.2.1 Input data processing

We derived a bare earth DEM and DSM with buildings and vegetation from a lidar survey acquired in 2013. Then we CNC routed 1:1500 meter scale models of the DEM, the DSM, and their inverses (see 3.4.4). We used the inverse models as molds for casting polymeric sand models.

We created the friction map – a required input for computing the cost surface – by computing the average friction cost for each cell from a set of friction maps based on the recoded rasterized maps of buildings, hydrology, floodplains, roads, soils, and trails with *r.series* using the average function.

First we converted all of the necessary vector maps to raster. Before converting linear features we set the resolution to 3 meters in order to give these features reasonable widths.

```
g.region n=224134 s=223005 w=637732 e=639085 res=1

v.to.rast input=lake_raleigh output=lake_raleigh use=val
v.to.rast input=chancellors output=chancellors use=val
v.to.rast input=buildings output=buildings use=val
v.to.rast input=floodplain output=floodplain use=val
v.to.rast input=hydrology_areas output=hydrology_areas use=val
v.to.rast input=trail_areas output=trails use=val
v.to.rast input=roads output=roads use=val

g.region res=3
v.to.rast input=greenways output=greenways use=val
v.to.rast input=streets output=streets use=val
v.to.rast input=hydrology output=hydrology use=val
g.region res=1
```

In order to limit processing to publicly accessible areas we made a composite map of inaccessible areas by combining maps of private land, buildings, and the lake using *r.mapcalc*[2] and then created an inverted mask from this map.

```
r.mapcalc "masking = if((buildings ||| chancellors ||| \
    lake_raleigh), 1, null())"
r.mask -i raster=masking
```

Next, we recoded all maps as friction (time penalties - additional time it takes to walk one meter), and then overlaid the maps using *r.mapcalc* and *r.series*.

```
r.mapcalc "hydro = if(isnull(hydrology_areas ||| hydrology), \
    1, 10)"
r.mapcalc "flood = if(isnull(floodplain), 1, 10)"
r.mapcalc "paths = if(isnull(trails ||| greenways), 10, 1)"
r.mapcalc "transit = if(isnull(streets ||| roads), 1, 10)"

r.series input=flood,hydro,paths,transit,soils output=friction \
    method=average
r.colors -n map=friction color=ryg
r.mask -r
```

The resulting friction map is shown in Fig. 7.3.

7.2.2 Computing the trail using the least cost path

In our case study we computed least cost paths between waypoints to create a trail network. LCPs were computed for each combination of two points. The following code snippet shows how such a network can be generated. For each waypoint we called the function `trail` defined below, which computes a raster representing the cumulative cost (time) from this waypoint and then multiple least cost paths from the remaining waypoints.

```
def trail(elevation, friction, walk_coeff, lambda_,
    slope_factor, point_from, points_to, vect_paths):
   """Computes least cost path to multiple locations based on
       walking time"""
   gscript.run_command('r.walk', flags='k',
       elevation=elevation, friction=friction, lambda_=lambda_,
       walk_coeff=walk_coeff, slope_factor=slope_factor,
       start_coordinates=point_from,
       stop_coordinates=points_to, output='tmp_walk',
       outdir='tmp_walk_dir')

   for i in range(len(points_to)):
      gscript.run_command('r.drain', flags='d',
          input='tmp_walk', direction='tmp_walk_dir',
          output='tmp_drain', drain=vect_paths[i],
          start_coordinates=points_to[i])
```

[2] note that symbol | | | for logical OR ignores NULL values and treats them as logical falses

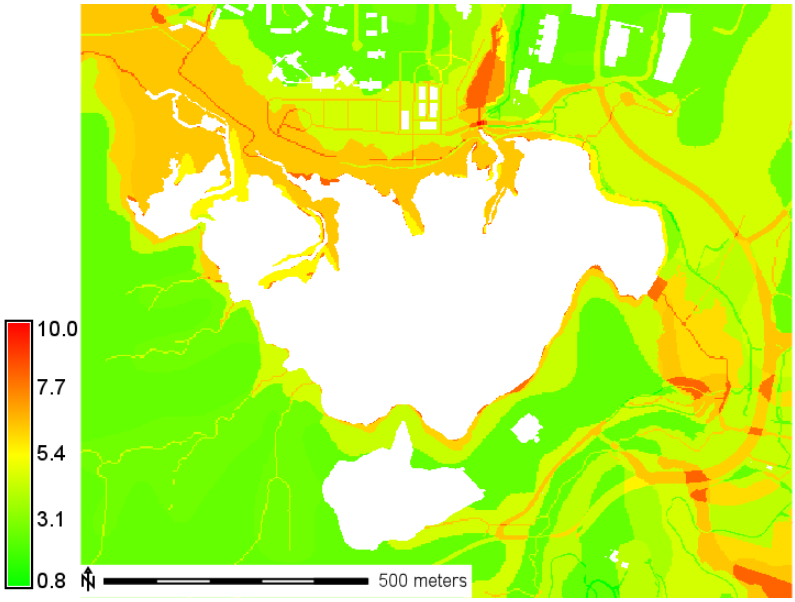


Fig. 7.3: The friction map [s/m] created by combining layers of buildings, hydrology, floodplains, roads, and trails.

```
  # remove temporary maps
  gscript.run_command('g.remove', type=['raster', 'vector'],
      name=['tmp_walk', 'tmp_walk_dir', 'tmp_drain'],
      flags='f')
```

This is repeated for all waypoints except the last.

```
def trails_combinations(elevation, friction, walk_coeff,
    lambda_, slope_factor, points, vector_routes):
   import itertools

   coordinates = gscript.read_command('v.out.ascii',
       input=points, format='point', separator=',').strip()
   coords_list = []
   for coords in coordinates.split():
      coords_list.append(coords.split(',')[:2])

   combinations = itertools.combinations(coords_list, 2)
   combinations = [list(group) for k, group in
       itertools.groupby(combinations, key=lambda x: x[0])]

   i = k = 0
   vector_routes_list = []
   for points in combinations:
      i += 1
```

```
        point_from = ',',.join(points[0][0])
        points_to = [',',.join(pair[1]) for pair in points]
        vector_routes_list_drain = []
        for each in points_to:
           vector_routes_list_drain.append('route_path_' +
               str(k))
           k += 1
        vector_routes_list.extend(vector_routes_list_drain)

        trail(elevation, friction, walk_coeff, lambda_,
            slope_factor, point_from, points_to,
            vector_routes_list_drain)

    gscript.run_command('v.patch', input=vector_routes_list,
        output=vector_routes)
```

The path, however, is only computed for combinations that were not covered in previous steps in order to reduce computation time. This means that the LCP is only computed in one direction. The direction matters because the speed of walking uphill and downhill differ. Therefore, the walking coefficients and slope factor of *r.walk* should be adjusted to affect uphill and downhill speed equally. This can be achieved by setting the slope factor to zero and using the same absolute number for coefficients b and d.

7.2.3 Finding the optimal trail

Using least cost path analysis we created all of the combinations of routes between waypoints. This can result in a large number of connections and potential trails. Therefore we used network analysis to find a trail loop that goes through all our waypoints in an optimal order avoiding connections with high costs. The solution for this well-known optimization problem – the Traveling Salesman Problem (TSP) – is implemented in the module *v.net.salesman*, which uses a heuristic algorithm yielding a good solution in reasonable time.

All GRASS GIS network analysis modules require a network vector map containing connected lines (arcs) and points (nodes). This vector map can be prepared using the module *v.net* which topologically connects the given points (our waypoints) and lines (our trail combinations). Since we wanted to route the trail through all our waypoints we found the categories of all waypoints in the network and then passed them to module *v.net.salesman*:

```
def trails_salesman(trails, points, output):
   gscript.run_command('v.net', input=trails, points=points,
       output='tmp_net', operation='connect', threshold=10)
   cats = gscript.read_command('v.category', input='tmp_net',
       layer=2, option='print').strip().split()
   gscript.run_command('v.net.salesman', input='tmp_net',
       output=output, center_cats=',',.join(cats), arc_layer=1,
       node_layer=2)
   # remove temporary map
```

```
    gscript.run_command('g.remove', type='vector',
        name='tmp_net', flags='f')
```

The final computation combines above defined functions and takes the scanned DEM and the detected points as input:

```
trails_combinations('scan', friction='friction',
    walk_coeff=[0.72, 6, 0, 6], lambda_=0.5, slope_factor=0,
    points='markers', vector_routes='route_net')
trails_salesman(trails='route_net', points='markers',
    output='route_salesman')
```

Examples of final output are shown in Fig. 7.1 and 7.4.

7.2.4 Mapping trail slopes

The simplest way to map the slope of a trail is to rasterize the trail using *v.to.rast* and then use *r.mapcalc* to extract the slope values from a slope raster computed with the module *r.slope.aspect*:

```
r.slope.aspect elevation=elevation slope=slope
v.to.rast input=trail output=raster_trail type=line use=cat
r.mapcalc "trail_slope = if(raster_trail, slope)"
```

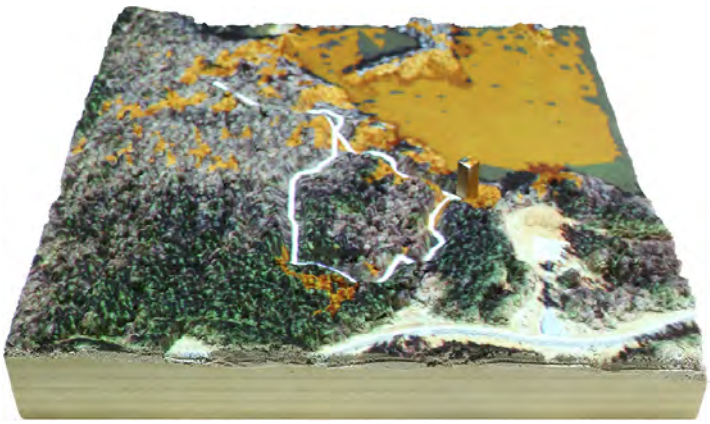
This extracts the slope values in the direction of the steepest slope. To compute the slope in the direction of the trail we first need to determine the direction of the trail by rasterizing its vector representation in terms of flow direction. The slope along the trail can then be computed as the slope multiplied by the cosine of the angle between the direction of the trail and the steepest slope (i.e. aspect):

```
r.slope.aspect elevation=elevation slope=slope aspect=aspect
v.to.rast input=trail output=raster_trail_dir type=line use=dir
r.mapcalc "trails_slope_dir = abs(atan(tan(slope) * cos(aspect \
    - raster_trail_dir)))"
```

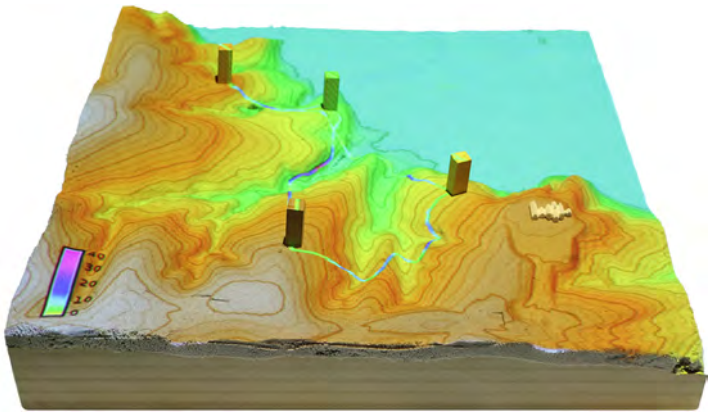
7.2.5 Alternative trail scenarios

We designed the initial trail scenario for Lake Raleigh Woods by placing markers at key points of interest – potential trailheads and scenic viewpoints (for more information about marker detection see Section 6.2.2). The siting of trailheads was informed by a previous 1 : 2500 meter scale study of potential routes around the entire lake (Fig. 7.6). We used realtime viewshed analysis to evaluate potential viewpoints (Fig. 7.5a). To test the viewsheds we cast the DSM in polymeric sand, carved clearings in the woodland canopy for viewpoints, placed markers to designate the viewpoints, and computed the viewsheds. We also computed the slope of along the trail so that we could critique the routes (Fig. 7.5b).

Fig. 7.4: A trail around Lake Raleigh



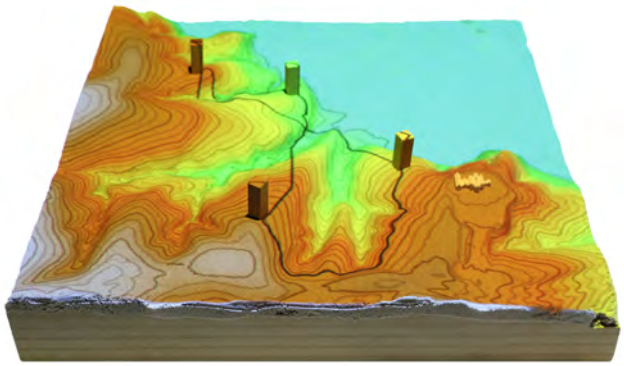
(a) The viewshed from a waypoint with areas visible represented in orange



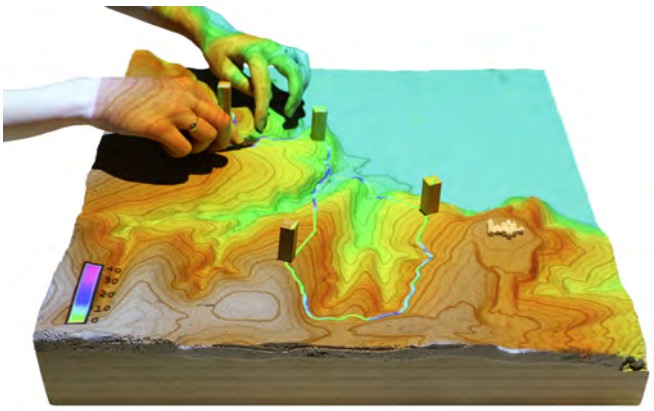
(b) The slope of the trail

Fig. 7.5: Trail analytics

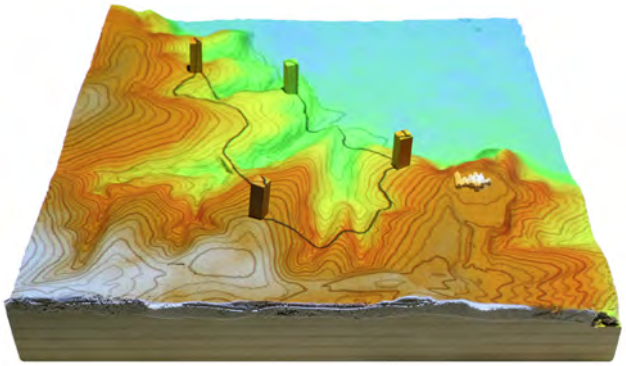
Our trail needed to cross three large gullies with steep slopes. In the initial trail scenario the route follows the contours and ridge lines to avoid the gullies with their challenging slopes. This route formed two loops that intersected at the mouth of the central gully near the lakefront with moderately steep slopes (Fig. 7.6a). We, however, wanted to experiment and see what a route with a single loop would look like so we built bridges across the gullies (Fig. 7.6b). The trail rerouted across the bridges to form a single, shorter loop with a lower average slope (Fig. 7.6c).



(a) The initial route for a trail in Lake Raleigh Woods



(b) Building bridges to reroute the trail across gullies with steep slopes



(c) The updated route for the trail across the new bridges

Fig. 7.6: Iteratively designing trail scenarios for Lake Raleigh Woods

References

Aitken, R. (1977). *Wilderness Areas in Scotland*. PhD thesis, University of Aberdeen.

Blank, G., Rudder, C., Dombrowski, A., Cser, H., Lawler, M., Kollar, C., Shifflett, S. D., Fortney, J., Sullivan, R., and Canavin, J. (2010). General Management Plan for Lake Raleigh Woods. Technical report, College of Natural Resources, North Carolina State University, Raleigh.

Fontanari, S. (2001). Svilluppo di metodologie gis per la determinazione dell'accessibilità territoriale come supporto alle decisioni nella gestione ambientale. Master's thesis, University of Trento.

Harmon, B. A. and Viles, H. (2013). Beyond geomorphosites: trade-offs, optimization, and networking in heritage landscapes. *Environment Systems and Decisions*, 33(2):272–285.

Langmuir, E. (1984). Mountaincraft and leadership. The Scottish Sports Council, Edinburgh.

Naismith, W. W. (1892). Untitled. Scottish Mountaineering Club Journal.